

Interview Research

Talent Assessment

[Talent Assessment Home](#)

[Interview Research Home](#)

[On-Site Research](#)

[Partners](#)



Interview Calibration Guides

- Access the [general information page](#) for the interview calibration guides.
- Access the [Leadership Principle guides page](#) to view LP guides
- Access the [functional competency guides page](#) to view guides for role-specific competencies

SDEI Guides (In Hire)

Coding (Logical and Maintainable)

Write syntactically-correct code that is logical, well-organized, easy to understand, works as intended, and is maintainable.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Creates overly complex code (e.g., contains improper coding constructs, doesn't consider reuse, poorly formatted) • Creates difficult to maintain code (e.g., complex if statements, un-			• Creates simple code (e.g., algorithm is broken out by logical functions, leverages reuse, code formatted consistently, no improper coding constructs) • Creates maintainable code (e.g.,	

naming conventions) • Code contains pseudo-code, or code does not work for basic use cases • Code doesn't work as intended		clear variable naming conventions, clear operations) • Syntactically correct code, or code that would be with minor edits • Code works as intended
--	--	--

Coding (Data Structures and Algorithms)

Uses optimal data structures and algorithms to solve problems, taking into account tradeoffs, limitations, and constraints.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Unable to identify data structures or algorithms to solve the problem • Unable to justify data structures and algorithms • Unable calculate runtime and space complexity • Does not understand shortcomings and is not able to communicate trade-offs of different data structures and algorithms (e.g., hashmaps, algorithms)			• Uses effective data structures and algorithms to solve the problem • Justifies why the selected data structures and algorithm was used • Demonstrates understanding of time and space complexities for the DS/algorithms used and can calculate it for the code written • Understands shortcomings and is able to communicate trade-offs of different data structures and algorithms (e.g., hashmaps, algorithms)	

Coding (Problem Solving)

Solve real-world coding problems by clarifying and precisely defining the problem and choosing the best approach, taking into account complexities, constraints, and tradeoffs.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Does not solve the problem • Begins solving the problem without asking relevant clarifying questions about the scope • Does not justify decisions with regard to technical requirements			• Solves the problem meeting all given requirements • Defines the scope of the problem by asking relevant clarifying questions • Provides justification for decisions with regard to technical requirements; shows an understanding of why a solution addresses requirement	

back to see solutions to the problem • Unable to change approach based on changes to the problem		lem • Able to change approach based on changes to the problem
--	--	---

SDEII Guides (In Hire)

Coding (Logical and Maintainable)

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Creates overly complex code (e.g., contains improper coding constructs, doesn't consider reuse, poorly formatted) • Creates difficult to maintain code (e.g., complex if statements, unstructured (e.g., spaghetti) code, obscure operations, poor variable naming conventions) • Code contains pseudo-code, or code does not work for basic use cases • Code doesn't work as intended			• Creates simple code (e.g., algorithm is broken out by logical functions, leverages reuse, code formatted consistently, no improper coding constructs) • Creates maintainable code (e.g., breaking inner loops into functions, breaking complex if statements, clear variable naming conventions, clear operations) • Syntactically correct code, or code that would be with minor edits • Code works as intended	

Coding (Problem Solving)

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Does not solve the problem • Does not progress past brute force solutions • Does not justify decisions and does not compare different solutions • Begins solving the problem			• Correctly solves the problem • Reaches an optimized solution • Justifies decisions by identifying potential tradeoffs with several different solutions; understands why one solution is better than another • Defines the problem by asking relevant	

• Commits to an initial solution without explaining tradeoffs with different solutions • Requires excessive hints or guidance to see solutions to the problem		• Requires minimal hints and is able to use these hints to reach solutions to the problem
---	--	---

Coding (Data Structures and Algorithms)

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Cannot identify data structures or algorithms to solve the problem • Does not identify potential shortcomings and discuss tradeoffs with different data structures and algorithms • Cannot justify data structures and algorithms • Cannot calculate runtime and space complexity or even recognize tradeoffs and optimizations			• Uses optimal data structures and algorithms to solve the problem • Identifies potential shortcomings and discusses tradeoffs with different data structures and algorithms • Justifies why the selected data structures and algorithm was used • Demonstrates solid grasp of runtime and space complexity tradeoffs even if not perfectly accurate O(n) syntax	

System Design

Amazon SDElls design software solutions to enable new features or improve their team's software. SDElls must solve difficult problems arising from visible risks or roadblocks. They make technologically appropriate decisions in the context of the broader business and technology strategy they work within. Their design decisions consider load and usage patterns and future needs.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Fails to validate given functional requirements within context. Does not capture basic customer use cases, scaling considerations, or other non-			• Identifies all requirements necessary for a working solution and clarifies requirements as needed. Identifies edge cases and articu-	

effective in the context of the problem.

- Does not clarify scope of design, customer behaviors, and/or other constraints, when directly asked to do so. Unable to break down the problem into functional components.
- Solution does not meet core functional requirements within problem constraints. Does not consider impact on other components in the system. Solution introduces significant and/or reasonably avoidable complexity.
- Solution may not function reliably under changes in utilization or other environmental changes. Solution creates avoidable maintenance challenges.
- Fails to identify tradeoff opportunities. Does not consider shortcomings of proposed solution, or alternative approaches.

- Identifies key technical decisions that enable a working feature. Asks questions and makes reasonable assumptions to identify boundaries and constraints of the problem.
- Solution meets identified requirements. Solution makes efficient use of resources in the given tech stack. Solution incorporates elements of fault-tolerance and/or monitoring.
- Solution is extensible to reasonable future changes in load and/or environment. Solution minimizes maintenance effort and costs. Solution considers operational performance under adverse conditions.
- Solution incorporates intentional tradeoff decisions in support of customer requirements. Candidate articulates the cases for and against different approaches.

SDEIII Guides (In Hire)

SDEIII Interviewing Overview



Coding (Logical and Maintainable)

Write syntactically-correct code that is logical, well-organized, easy to understand, works as intended, and is maintainable.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Creates unnecessarily complex code (e.g., monolithic functions, complex function signatures, improper coding constructs) - Creates difficult to maintain code (e.g., lacks code reuse, improper use of globals, poor variable naming conventions) - Code is organized in a way that is difficult to read and understand (e.g., does not encapsulate, code is not formatted, variables naming unclear) - Does not have sufficient consideration for extensibility - Writes code that is not easy to test or debug			- Creates simple code (e.g., algorithm is broken out by logical functions, leverages reuse, code formatted consistently, no improper coding constructs) - Creates maintainable code (e.g., leverages reuse, helper functions, clean boundaries between functions or modules, uses language features appropriately) - Code is organized in a way that is easy to read and understand (e.g., encapsulates appropriately, clear variable naming conventions) - Identifies areas where requirements may evolve, and addresses with an extensible design - Writes code that is easy to test and debug	

Show Sample Coding(Logical and Maintainable) questions

Coding (Problem Solving)

Solve real-world coding problems by clarifying and precisely defining the problem and choosing the best approach, taking into account complexities, constraints, and tradeoffs.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Solves a simpler or only part of presented problem and cannot improve or expand when prompted • Does not consider alternatives or take hints; does not respond to feedback • Begins solving the problem without asking relevant clarifying questions and starts coding before clearly describing the solution • Stays at a high level, dismisses concerns about details, or does not code any portion • Does not commit to an approach; relies on interviewer for implementation decisions • Solutions are fragile with respect to bad data, edge cases, or errors, and does not consider impact of errors on customers or other components			• Correctly solves the problem by choosing an optimal approach from a variety of solutions; explains choices clearly and succinctly • Requires minimal or no hints and is able to use these hints to reach solutions to the problem • Clarifies ambiguity by considering business or customer concerns and proposes alternatives to meet different scalability constraints • Considers additional factors beyond CS fundamentals (e.g., developer effort, team composition, customer impact, shortand long-term business needs) • Applies algorithmic complexity for both worst and average cases and knows when an optimal solution is reached • Creates solutions that are simple, robust, will handle edge cases and changes in circumstances	

Coding (Data Structures and Algorithms)

Uses optimal data structures and algorithms to solve problems, taking into account tradeoffs, limitations, and constraints.

Concern	Mild Concern	Mixed	Mild Strength	Strength
• Unable to identify data structures or algorithms to solve the problem • Unable to justify data structures and algorithms			• Uses effective data structures and algorithms to solve the problem • Justifies why the selected data structures and algorithm was used	

Does not understand shortcomings and is not able to communicate trade-offs of different data structures and algorithms (e.g., hashmaps, algorithms)		the DS/algorithms used and can calculate it for the code written Understands shortcomings and is able to communicate trade-offs of different data structures and algorithms (e.g., hashmaps, algorithms)
---	--	---

System Design

Develop functional and non-functional requirements by understanding and clarifying assumptions, needs, and system dependencies, taking into account tradeoffs in terms of scalability, maintainability, extensibility, performance, and security.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Fails to clarify requirements, make reasonable assumptions, or manage ambiguity - Fails to describe a viable, coherent, and complete design that fulfills requirements - Does not consider or measure performance or operational excellence - Fails to contemplate multiple design choices or independently make the selection of the path forward - Does not employ existing constructs to simplify their design and includes complex or unnecessary components			- Removes ambiguity by taking problem statement and further identifying implicit requirements - Drives the design discussion by showing logical progression from problem statement to multiple components, each with clear roles and responsibilities, while working backwards from the customer - Design extends beyond the base functional needs, looking at other dimensions such as performance, security, scaling, operations, etc. - Identifies potential shortcomings and tradeoffs with different designs, and chooses a reasonable path that is evolvable (e.g., performance, reliability, complexity) - Recognizes and discusses the use of design patterns and principles, or existing technology and industry standards within their domain of expertise	

Coding (Logical and Maintainable)

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Creates unnecessarily complex code (e.g., doesn't consider reuse, poorly formatted, contains improper coding constructs) - Creates difficult to maintain code (e.g., difficult to trace impact of changes, poor variable naming conventions) - Code is organized in a way that is difficult to read and understand - Code contains major syntax errors or is pseudo-code - Code doesn't work as intended			- Creates simple code (e.g., leverages reuse, properly formatted, no improper coding constructs) - Creates maintainable code (e.g., quickly able to trace impact of changes, clear variable naming conventions) - Code is organized in a way that is easy to read and understand - Code is syntactically correct, or would be syntactically correct with minor improvements - Code works as intended	

Front-end Technology

Amazon FEEs have a solid understanding of the fundamental technologies used to render user interfaces and how these technologies are used collectively to render experiences, respond to app and view lifecycle events, and manage application state. Even when abstracted by frameworks, FEEs understand how their code is transpiled to lower level fundamentals, and best practices for these fundamentals. Depending on the target platform, these fundamentals may include HTML/JS/CSS, or current industry standard native UI systems.

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Cannot describe accessibility concepts and strategies - Unable to make the solution support more than a single customer's needs (e.g. screen size, device/browser compatibility, internationalization) - Does not follow industry standard best practices, or does not justify deviation			- Creates an accessible solution, or speaks in depth about strategies that would create an accessible experience - Demonstrates an ability to create a solution that will address a wide variety of customer needs and situations (e.g. device and browser support, internationalization/localization, bandwidth concerns) - Leverages industry standards to create a	

derstanding of the underlying platform for their solution. (e.g. makes incorrect assumptions about the framework, how events or state are processed and managed) Does not create a logical separation between available components (e.g. view, style, behavior, data modelling, business logic)		ence - Implements a solution that is optimized for scale or reuse (e.g. does not violate system constraints, such as reusing ID attributes in HTML document; number of repaints scales linearly with respect to data or network traffic; network traffic scales linearly with regards to user interaction frequency) - Identifies potential shortcomings and discusses trade-offs in the approach and technologies used - Describes testing practices that are practical and able to catch issues which may be found in any part of the solution
--	--	---

Coding (Data Structures and Algorithms)

Concern	Mild Concern	Mixed	Mild Strength	Strength
- Cannot identify data structures or algorithms to solve the problem - Does not identify potential shortcomings and discuss tradeoffs with different data structures and algorithms - Cannot justify data structures and algorithms - Cannot calculate runtime and space complexity or even recognize tradeoffs and optimizations			- Uses optimal data structures and algorithms to solve the problem - Identifies potential shortcomings and discusses tradeoffs with different data structures and algorithms - Justifies why the selected data structures and algorithm was used - Demonstrates solid grasp of runtime and space complexity tradeoffs even if not perfectly accurate $O(n)$ syntax	

System Design

Concern	Mild Concern	Mixed	Mild Strength	Strength
---------	--------------	-------	---------------	----------

- Fails to fulfill requirements (e.g., constraints, scalability, maintenance)
- Does not consider or measure operational performance
- Does not discuss potential shortcomings or tradeoffs with different designs (e.g., performance, fault tolerance, dependencies)

- down and define requirements
- Creates a design for a system that fulfills captured requirements (e.g., constraints, scalability, maintenance)
- Designs for operational performance and plans for failure and can measure the results (e.g., metrics)
- Identifies potential shortcomings and tradeoffs with different designs (e.g., performance, fault, tolerance, dependencies)



Where are the rest?

Where is the guide for my job?

Functional competencies have been created for a subset of jobs using [Project Calibrate](#). This is a customer-driven process. If your job is not listed that means the work group has not been formed or is still in progress. You can determine which by going looking up your job family [here](#).

How can I pitch in?

Look up your job family [here](#). If there is a job family owner already, reach out to him or her. If there isn't, put on your Ownership hat and follow the three steps outlined [here](#) to create and submit a guide to Talent Assessment.

